

2026년 오픈소스 개발자대회 결과보고서

과제 및 팀 정보

팀 명	기입 필요	참가부문	학생 / 일반
팀 인원	팀장 포함 명	과제유형	지정과제 (리원에이스)

프로젝트 개요

프로젝트명	MCP 기반 지능형 데이터 플랫폼
프로젝트 등록 URL	https://github.com/HyunBeenL/liwonaceProject
시연 영상	YouTube 링크 기입 필요
프로젝트 소개	일상어 질문을 규칙 기반 라우터가 분석해 적합한 MCP 도구로 위임하고, PostgreSQL 조회 결과를 로컬 소형 LLM이 자연어로 정리해 답변하는 사내 데이터 질의 플랫폼

프로젝트 세부 내용

개발 배경 및 목적

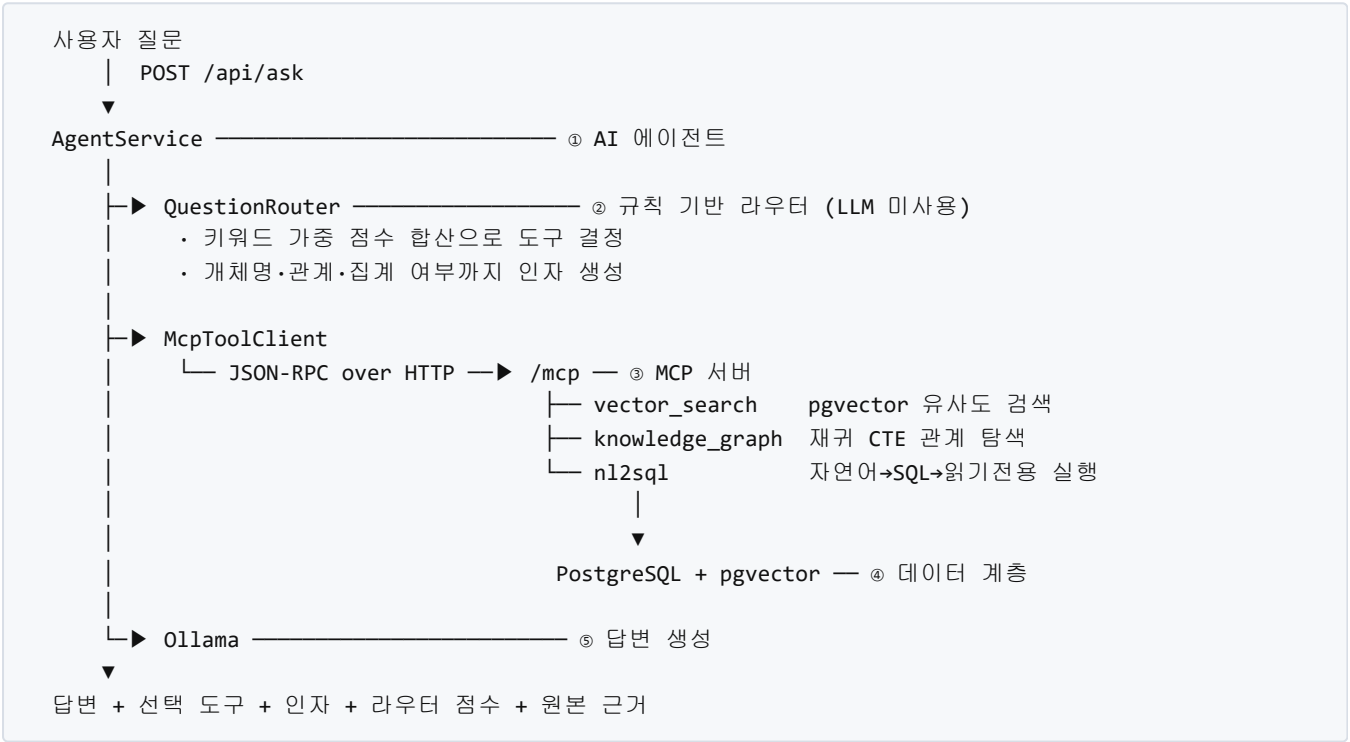
- 기존 RAG 파이프라인의 운영 복잡도 문제
 - 청킹 → 임베딩 → 인덱싱 → 검색 → 리랭킹의 각 단계마다 튜닝 파라미터가 존재
 - 단계 간 상호 영향으로 설정 하나가 어긋나면 정확도가 급락하고, 장애 지점이 분산됨
- MCP(Model Context Protocol)를 통한 단순화
 - 도구 호출을 표준 프로토콜 한 겹으로 통합해 파라미터와 장애 지점을 축소
 - 표준을 따르므로 특정 클라이언트에 종속되지 않음
- 소형 로컬 LLM으로 실용적 정확도 달성
 - 외부 API 없이 Ollama 로컬 실행만으로 구성 (데이터 외부 유출 없음)
 - 소형 모델이 취약한 판단은 규칙으로 분리하고, 모델은 잘하는 일에만 투입
- 달성 목표
 - 세 가지 데이터 형태(비정형 문서 / 정형 테이블 / 개체 관계)를 하나의 질의 창구로 통합
 - 어떤 도구를 왜 선택했는지 설명 가능한 구조

개발 환경

- 언어 및 런타임 — Java 21, Spring Boot 4.1.0, Spring AI 2.0.0, Maven(mvnw 래퍼 포함으로 별도 설치 불필요)
- 데이터 저장소 — PostgreSQL 17 + pgvector 확장(HNSW 코사인 인덱스), Docker Compose 구성 및 초기화 SQL 자동 실행
- AI 모델(전량 로컬 실행) — 임베딩 bge-m3 (1024차원), 언어모델 gemma4:e2b, 실행 런타임 Ollama
- 개발 도구 — IntelliJ IDEA, Git/GitHub, 검증 스크립트 PowerShell 5종, 테스트 JUnit 5(라우터 정확도 자동 채점)
- 검증 환경 — CPU 12코어 / RAM 31.7GB, GPU AMD Radeon RX 7600(ROCm, 선택 사항)

시스템 구성 및 아키텍처

시스템 구성도



기술 스택 및 역할

계층	기술	역할
진입점	Spring MVC	질문 수신, 정적 시연 페이지 제공
에이전트	자체 구현 (AgentService)	전체 흐름 제어, 폴백:재시도 판단
라우터	자체 구현 (QuestionRouter)	도구 선택 및 인자 생성 (결정적)
프로토콜	MCP Streamable HTTP	에이전트↔도구 간 표준 통신
도구	Spring AI @Tool 3종	검색 · 관계탐색 · SQL생성
저장소	PostgreSQL 17 + pgvector	벡터 · 정형 · 그래프 데이터 통합 저장
모델	Ollama (bge-m3, gemma4:e2b)	임베딩, SQL 변환, 답변 생성

핵심 데이터 흐름

- **질문 → 도구 결정**: 라우터가 관계 키워드(4점), 문서:정형 키워드(3점), 개체명:수량 표현(2점)을 합산해 최고 점 도구를 선택. 동일 점수를 판단 신뢰도로도 재사용
- **도구 → 조회**: 도구별로 임베딩 검색 / 재귀 CTE / LLM SQL 생성 후 읽기 전용 실행
- **조회 → 답변**: 확보한 근거만을 컨텍스트로 주어 LLM이 문장 생성(근거 외 내용 생성 금지)
- **적재 흐름**: 기동 시 문서 40건을 임베딩해 `document_chunks` 에, 그래프 JSON을 `graph_nodes` / `graph_edges` 에 적재(테이블이 비어 있을 때만 수행)

프로젝트 주요 기능

프로젝트 상세 내용

핵심 기능 1 — 규칙 기반 도구 자동 선택

- 질문 유형을 키워드 가중 점수로 판별해 세 도구 중 하나로 위임
- 첫 매치가 아닌 **점수 합산 방식**을 채택. 동일 표현이 도구를 가로질러 등장하기 때문
 - "가장 많은"은 3개 문항에서 2개 도구로 분기 → 관계 키워드 동반 여부로 판별
 - "이슈"는 관계 신호이나 "미팅에서 논의된 이슈"는 회의록 검색 → 문서 신호 누적으로 역전
 - "Product-C1"은 개체명이나 "설치 방법"과 결합 시 기술문서 질의
- 도구 이름뿐 아니라 **인자까지 생성**: 개체명 추출, 관계 추론, 다중 홑 판단, 집계 여부
- 판단 근거(도구별 점수·기여 키워드)를 응답에 포함해 설명 가능성 확보

핵심 기능 2 — MCP 도구 3종

도구	처리 방식	대상 데이터
vector_search	bge-m3 임베딩 후 코사인 거리 검색, 유사도 하한 0.5	비정형 문서 40건
knowledge_graph	재귀 CTE 양방향 탐색, 다중 홑·관계 집계 지원	노드 133 · 관계 354
n12sql	LLM이 SQL 생성 후 3중 검증 하에 실행	8개 테이블 818행

핵심 기능 3 — LLM이 생성한 SQL의 3중 안전장치

모델이 생성한 문자열을 DB에 전달하는 구조이므로 방어를 계층화하였습니다.

- 구문 검사** — `SELECT / WITH` 외 거부, 조회 외 키워드 차단, 다중 문장 차단. 주석·문자열 리터럴을 제거한 뒤 검사해 값에 의한 오탐 방지
- 읽기 전용 트랜잭션** — 구문 검사를 통과해도 쓰기 연산이 실패
- 실행 시간·행 수 제한** — `SET LOCAL statement_timeout`, 최대 100행

핵심 기능 4 — 규칙과 모델의 협업 분기

- 규칙이 **확신하는 구간**(최고점 > 0 이고 2위와 동점 아님)은 규칙대로 처리
- 확신하지 못하는 구간**만 LLM에게 도구 선택 위임. 단 인자는 여전히 규칙이 생성
- 도구가 **빈손으로 반환**하면 다음 순위 도구로 1회 재시도
- 경계값은 예시 질문 30문항의 실측 분포(최소 1위 2점, 최소 격차 1점)보다 낮게 설정하여 검증된 경로를 침범하지 않도록 함

개발 과정

단계	내용
1	인프라 구성 — Docker Compose, 초기화 SQL, 스키마 확장
2	데이터 적재 — 문서 임베딩, 그래프 JSON 적재
3	MCP 도구 3종 구현 및 도구별 검증
4	규칙 기반 라우터 구현 및 30문항 자동 채점
5	에이전트 구현 — MCP 클라이언트 연결, 답변 생성
6	성능 측정 및 최적화 — 추론 옵션 분리, GPU 전환
7	일반화 측정 및 협업 분기 추가

각 커밋은 독립적으로 컴파일되며, 설계 판단의 근거를 커밋 메시지에 기록하였습니다.

구동 및 시연

실행 방법

```
# 1. 컨테이너 기동 후 healthy 확인
docker compose up -d
docker compose ps

# 2. 모델 준비 (최초 1회, 약 7.7 GiB)
docker exec reaone-ollama ollama pull bge-m3
docker exec reaone-ollama ollama pull gemma4:e2b

# 3. 데이터셋을 dataset/ 에 배치
#   (대회 제공 데이터셋은 재배포 조건상 저장소에 미포함)

# 4. 실행
./mvnw spring-boot:run
```

브라우저에서 `http://localhost:8080` 접속 시 질문 입력, 선택된 도구, 라우터 점수, 답변, 원본 근거를 한 화면에서 확인할 수 있습니다.

테스트 방법

```
./mvnw test
```

- **컨테이너 없이 실행 가능.** 라우터의 도구 선택 정확도를 예시 질문 30문항으로 자동 채점
- DB가 필요한 통합 테스트는 `integration` 태그로 분리 (`./mvnw test -Dsurefire.excludedGroups=` 로 포함 실행)

검증 결과

항목	결과
라우터 도구 선택 (예시 질문 30문항)	30 / 30
vector_search 문서 타입 적중	10 / 10
knowledge_graph	9 / 10 (1문항은 데이터셋 자체 결함)
n12sql 실행 성공 / 정답	10 / 10 · 9 / 10 안팎

응답 시간 (질문 → 답변 전 구간)

도구	CPU	GPU (ROCm)
knowledge_graph	4.8초	0.9초
vector_search	18.3초	3.7초
n12sql	34.5초	8.4초

기대효과 및 활용 분야

향후 확장성

- **도구 추가가 독립적** — MCP 도구를 추가하고 라우터에 키워드를 등록하면 확장 완료. 기존 도구·에이전트 코드 수정 불필요
- **데이터 소스 교체 용이** — 도구 내부만 교체하면 되므로 사내 위키, 이슈 트래커, 파일 서버 등으로 대상 확장 가능
- **모델 독립성** — Ollama가 지원하는 모델로 자유 교체. 작업별로 다른 모델 배정 가능 (예: SQL 생성은 코드 특화 모델, 답변 생성은 경량 모델)
- **표준 클라이언트 호환** — MCP 표준을 구현했으므로 별도 개발 없이 다양한 MCP 클라이언트에서 사용 가능

기대효과

- **데이터 주권 확보** — 전 과정 로컬 실행으로 사내 데이터가 외부로 전송되지 않음. 보안 규정이 엄격한 조직에서도 도입 가능
- **운영 비용 절감** — 외부 API 호출 비용 없음. 기존 사내 서버에서 구동 가능
- **판단 근거의 추적성** — 도구 선택 점수와 원본 근거가 응답에 포함되어 오답 발생 시 라우터·도구·모델 중 어느 단계의 문제인지 즉시 구분 가능

활용 분야

- 사내 지식 검색 (규정 · 매뉴얼 · 회의록 통합 질의)
- 운영 데이터 셀프서비스 조회 (비개발자의 SQL 없는 데이터 접근)

- 고객 지원 이력 및 관계 분석
- 폐쇄망 환경의 문서 · 데이터 질의 시스템

프로젝트의 혁신성 및 차별성

1. 규칙과 소형 LLM의 역할 분담을 실측으로 설계

단순히 규칙을 쓴 것이 아니라, 어느 작업을 규칙에 맡기고 어느 작업을 모델에 맡길지를 측정으로 결정했습니다. 세 방식을 42문항으로 비교한 결과는 다음과 같습니다.

방식	예시 질문 30	표현 변경 12	합계
규칙만	30 / 30	8 / 12	38 / 42
LLM만	24 / 30	10 / 12	34 / 42
규칙 + LLM 협업	30 / 30	10 / 12	40 / 42

LLM 단독은 표면 단어에 이끌리는 오류를 보였습니다("DB 튜닝 방법" → `n12sql` 선택). 규칙은 도메인 어휘가 일치할 때 강하고, LLM은 표현이 바뀌어도 의미를 파악합니다. **규칙의 확신도가 두 방식의 강점 경계와 일치했**기에, 이를 분기 기준으로 삼아 양쪽 단독보다 높은 정확도를 얻었습니다.

2. 추론(thinking) 옵션을 작업별로 분리

동일 모델의 추론 옵션이 작업 성격에 따라 정반대 효과를 냈습니다.

작업	추론 켜	추론 끄	채택
SQL 생성	9 / 10 정답	5 / 10 정답	켜
답변 생성	39.7초 / 671토큰	14.3초 / 147토큰	끄

SQL 생성에서 추론을 끄면 별칭 불일치(`T1`로 조인 후 `T2` 참조), 미존재 컬럼 호출이 발생했습니다. **조인과 별칭을 맞추는 것이 추론이 수행하던 작업**이었습니다. 반면 답변 생성은 끄는 쪽이 더 빠르면서 더 정확했습니다. 추론을 켜면 "장애가 발생했습니다" 수준으로 추상화되어 고유명사가 소실되지만, 끄면 근거의 고유명사를 그대로 인용했습니다.

소형 모델의 추론 옵션은 켜고 끄는 문제가 아니라 작업별로 분리하는 문제라는 결론을 실측으로 도출했습니다.

3. 오답의 발생 지점을 구분할 수 있는 응답 구조

응답에 선택 도구, 전달 인자, 라우터 점수, 도구가 반환한 원본 근거를 모두 포함합니다. 답변이 틀렸을 때 라우터의 오선택인지, 도구의 조회 실패인지, 모델의 요약 오류인지 즉시 구분됩니다. 실제로 "최근 서버 장애" 질의에서 시간 조건이 반영되지 않은 원인이 벡터 검색에 시간 정렬이 없기 때문임을 이 구조로 특정했습니다.

4. 검증 가능한 SQL 생성

`n12sql` 도구는 실행한 SQL을 결과와 함께 반환합니다. 초기 검증에서 10문항 모두 "실행 성공"이었으나 SQL을 개별 확인한 결과 2건이 오류였습니다. 그중 하나는 `employees.dept_id = departments.head_id` 라는 잘못된

조인이었는데, 해당 데이터에서 head_id 와 id 가 우연히 일치해 **결과만으로는 발견할 수 없는 오류**였습니다. 이를 계기로 "조인은 외래키 목록에 존재하는 관계로만 수행" 규칙을 프롬프트에 추가했습니다.

한계점 및 향후 발전 로드맵

현재 한계

한계	내용	보완 방안
도메인 어휘 의존	표현이 바뀌면 규칙 정확도 8/12로 하락	협업 분기로 10/12까지 회복. 어휘 사전 확충 또는 임베딩 기반 유사도 매칭 도입
시간 조건 미처리	벡터 검색에 시간 정렬 부재로 "최근" 반영 불가	메타데이터 날짜 기반 후처리 정렬 또는 라우터의 시간 표현 감지
NL2SQL 정확도 변동	temperature 0에도 실행마다 오답 문항 변동	생성 SQL 검증 강화, 실패 시 재생성
무상태 질의	후속 질문("그 중에서...") 미지원	대화 컨텍스트 유지 계층 추가
단일 도구 호출	복수 도구 조합 질의 미지원	도구 결과 기반 연쇄 호출

향후 로드맵

- **단기** — 라우터 어휘 확충 및 협업 분기 정교화, 시간 조건 처리, 답변 환각률 정량 측정
- **중기** — 대화 컨텍스트 지원, 복수 도구 조합, 작업별 모델 분리 배정 (SQL 특화 모델 도입), 단순 집계는 템플릿 처리로 LLM 호출 절감
- **장기** — 도구 플러그인 구조화(신규 데이터 소스 추가 시 코드 수정 최소화), 운영 지표 수집(도구별 정확도·응답 시간 모니터링), 다중 사용자 환경 대응

유지보수 관점

- 라우터 정확도가 회귀 테스트로 고정되어 있어 규칙 수정 시 즉시 검증됨
- 컨테이너 없이 단위 테스트가 실행되어 기여자의 진입 장벽이 낮음
- 설계 판단의 근거를 커밋 메시지와 코드 주석에 기록해 이후 수정 시 맥락 파악 가능

소감 및 후기

직접 작성 필요 — 아래는 개발 과정에서 실제 있었던 사실입니다. 참고하여 작성하시기 바랍니다.

- **"실행 성공"과 "정답"이 다르다는 것을 확인한 경험** — NL2SQL 검증에서 10문항 모두 실행에 성공했으나 SQL을 개별 확인하자 2건이 잘못되어 있었습니다. 그중 하나는 데이터의 우연으로 결과까지 맞았기에, 결과만 보고 넘어갔다면 발견하지 못했을 오류였습니다.
- **문서화가 적은 최신 스택 조합에서의 문제 해결** — Spring Boot 4 + Spring AI 2.0 조합에서 MCP 전송 계층이 조용히 비활성화되는 문제(로그는 정상, 엔드포인트만 404)를 조건 평가 리포트로 원인을 특정했습니다. 또한 도구가 LLM을 주입받는 순간 순환 참조가 발생하는 구조적 문제를 발견하고 지연 해결로 우회했습니다.

- **성능 문제의 원인을 추측이 아닌 측정으로 규명** — 응답이 느린 원인을 프레임워크로 의심했으나, 구간별 측정 결과 45.34초 중 45.28초가 모델 추론 시간이었고 프레임워크 오버헤드는 0.06초였습니다. 이후 추론 옵션 분리와 GPU 전환으로 12배 개선했습니다.
- **한계를 측정한 것이 개선으로 이어진 경험** — 예시 질문에서 100%였던 라우터가 같은 의미의 다른 표현에서는 67%임을 확인하고, 이를 근거로 규칙과 모델의 협업 분기를 설계했습니다.

붙임1 SBOM (소프트웨어 자재명세서)

선별 기준: ① GPL·AGPL·LGPL 계열 → ② 핵심 기능 담당 → ③ 주요 프레임워크·SDK → ④ 빌드·실행 도구
전체 의존성 187개 목록은 저장소 `docs/DEPENDENCIES.txt` 참조(자동 생성)

번호	라이브러리명	버전	라이선스	공식 저장소 URL	사용 목적 및 주요 기능
1	Logback Classic	1.5.34	EPL-2.0 또는 LGPL-2.1-only (듀얼)	github.com/qos-ch/logback	애플리케이션 로깅 / 라이브러리로 불러 씸(미수정 링크)
2	Logback Core	1.5.34	EPL-2.0 또는 LGPL-2.1-only (듀얼)	github.com/qos-ch/logback	로깅 코어 모듈 / 라이브러리로 불러 씸(미수정 링크)
3	Jakarta Annotations API	3.0.0	EPL-2.0 또는 GPL-2.0 with Classpath Exception	github.com/jakartaee/common-annotations-api	표준 애노테이션 / 라이브러리로 불러 씸
4	Jakarta Transaction API	2.0.1	EPL-2.0 또는 GPL-2.0 with Classpath Exception	github.com/jakartaee/transactions	트랜잭션 표준 API / 라이브러리로 불러 씸
5	Java MCP SDK	2.0.0	MIT	github.com/modelcontextprotocol/java-sdk	MCP 프로토콜 서버-클라이언트 구현 / 라이브러리로 불러 씸
6	pgvector-java	0.1.6	MIT	github.com/pgvector/pgvector-java	벡터 타입 매핑 / 라이브러리로 불러 씸
7	PostgreSQL JDBC Driver	42.7.11	BSD-2-Clause	github.com/pgjdbc/pgjdbc	DB 연결 및 질의 실행 / 라이브러리로 불러 씸
8	Spring Boot	4.1.0	Apache-2.0	github.com/spring-projects/spring-boot	애플리케이션 프레임워크 / 라이브러리로 불러 씸
9	Spring AI	2.0.0	Apache-2.0	github.com/spring-projects/spring-ai	MCP 서버 노출, Ollama 연동 / 라이브러리로 불러 씸
10	Hibernate ORM	7.4.1.Final	Apache-2.0	github.com/hibernate/hibernate-orm	JPA 구현체 / 라이브러리로 불러 씸

라이선스 충돌 검토 결과

- 강한 카피레프트(GPL · AGPL)는 발견되지 않았습니다.

- 1~2번(Logback)은 EPL-2.0 선택이 가능한 듀얼 라이선스이며, 수정 없이 링크만 하므로 LGPL을 택하더라도 소스 공개 의무가 발생하지 않습니다.
- 3~4번은 GPL-2.0이 표기되어 있으나 **Classpath Exception**이 명시되어 링크하는 코드에 라이선스가 전파되지 않습니다. Java 표준 API의 통상적 라이선싱 형태입니다.
- 본 프로젝트 라이선스(Apache-2.0)와 충돌하는 항목은 없습니다.
- 대회 제공 데이터셋은 "대회 참가 목적으로만 사용 가능" 조건이므로 Apache-2.0 배포와 양립하지 않아 저장소에 포함하지 않았습니다.

붙임2 AI 모델 활용 및 라이선스 기술 명세서

1. AI 모델 활용 유형

☒	유형 1: 외부 모델 그대로 활용 (추가 학습 없이 기존 공개 모델을 프로젝트에 연동·구동)
☐	유형 2: 외부 모델 파인튜닝
☐	유형 3: 자체 개발 모델

2. 기반(베이스) 모델 정보

기반 모델명 및 개발사	① Gemma 4 E2B (Google) — 답변 생성 및 자연어→SQL 변환 ② BGE-M3 (Beijing Academy of Artificial Intelligence) — 문서·질의 임베딩	기반 모델 라이선스	① Gemma Terms of Use ② MIT License
--------------	--	------------	---------------------------------------

- 두 모델 모두 **가중치를 수정하지 않고** Ollama 런타임을 통해 로컬에서 구동합니다.
- 파인튜닝, 추가 학습, 가중치 병합을 수행하지 않았습니다.
- **확인 권장** 각 모델의 라이선스 표기는 배포처 공식 문서로 최종 확인 바랍니다.

3. 데이터셋 정보 및 가중치 배포 명세

해당 없음 (유형 1에 해당하여 학습 데이터셋 및 신규 가중치가 존재하지 않음)

4. 소스코드 라이선스 및 개발 환경 정보

직접 작성한 코드의 오픈소스 라이선스	Apache License 2.0 (OSI 인증)
소스코드 공개 저장소 URL	https://github.com/HyunBeenL/liwonaceProject (공개 유지)
상용 AI 보조도구 활용 여부 및 범위	코드 작성 및 디버깅 보조용으로 Claude(Anthropic) 활용. 설계 판단과 검증은 직접 수행하였으며, 제출 시스템에는 외부 AI API가 포함되지 않고 Ollama 로컬 모델만 사용함 ※ 활용 범위(비율 등)는 실제 작업 비중에 맞게 조정